

Lesson 1: Why React Exists and How Components Work

Mission Brief

Your Lumexa Explorer console has grown from a few buttons into a sprawling control panel with dozens of readouts, all updating at once. Hand-written `document.querySelector` code is starting to buckle under the weight — one missed update and the fuel gauge shows the wrong number. Today you meet **React**, the tool professional teams use to build interfaces that stay correct no matter how complex they get, by describing screens as reusable **components**.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain the problem React solves compared to manually updating the DOM
- Describe what a "component" is and why UIs are built out of them
- Create a functional React component and render it to the page
- Explain the relationship between the Virtual DOM and the real DOM at a conceptual level
- Set up and run a React project locally with Vite

What You'll Build

A small "Mission Control" React app with three simple components — a `Header`, a `StatusPanel`, and a `Footer` — composed together inside one root `App` component, running in a real Vite dev server.

Prerequisites

- Comfort with JavaScript functions, variables, and arrays (Course 08)
- Basic HTML and CSS (Course 07)
- Node.js and npm installed, and familiarity with running terminal commands

Key Concepts

- Imperative vs. declarative UI code
- Components as functions that return UI
- JSX as a preview (deep dive in Lesson 2)
- The Virtual DOM (conceptual)
- Composing components together

- Project setup with Vite

Concept Explanation

Before React, building an interactive page meant writing **imperative** code: step-by-step instructions telling the browser exactly how to find an element and mutate it — `document.querySelector("#count").textContent = newValue`. This works fine for small pages, but as an app grows to dozens of interacting pieces of state, it becomes very easy for the DOM to drift out of sync with your actual data, because there are so many individual update statements to keep track of and so many places a bug can hide.

React introduced a **declarative** model instead: instead of writing instructions for *how* to update the screen, you describe *what* the screen should look like for any given piece of data, and React figures out the most efficient way to make the real DOM match that description. You write a function that says "given this data, the screen looks like this," and whenever the data changes, you simply describe the new "this," and React handles the updating.

The unit of that description is a **component**. A component is just a JavaScript function whose job is to return a description of some UI. Components can be small (a single button) or large (an entire page), and — this is the important part — they can be **composed**: a big component is usually built out of several smaller components nested inside each other, exactly like nesting HTML elements, except each piece is a reusable, named building block with its own logic. A `ProductPage` component might be made of a `ProductGallery` component and a `ProductInfo` component, each of which might itself be made of even smaller components.

Under the hood, React keeps an in-memory representation of the UI called the **Virtual DOM** — a lightweight JavaScript description of what the real DOM should look like. When your data changes, React builds a new Virtual DOM tree, compares it to the previous one (a process called "diffing"), and applies only the minimal set of real DOM changes needed to match — rather than throwing away and rebuilding the entire page. You rarely think about the Virtual DOM directly day-to-day, but it's *why* React can be both declarative and fast.

Finally, every serious React project today is built with a **build tool** like Vite, which takes your component files, compiles the special JSX syntax (next lesson) into plain JavaScript, bundles everything together, and serves it with a fast local dev server that updates the browser instantly as you save files — a feature called Hot Module Replacement.

Guided Coding

First, a real project is created and run like this (do this once, in a terminal, outside of any file you write):

```
npm create vite@latest mission-control -- --template react
cd mission-control
npm install
npm run dev
```

That generates a real, runnable project. Inside it, here is what our three-component "Mission Control" looks like:

```
// src/components/Header.jsx

// A component is just a function that returns UI (JSX, which we formally
// cover in Lesson 2). By convention, component names start with a
// capital letter – this is how React tells components apart from
// regular HTML tags like <div> or <button>.
export default function Header() {
  return (
    <header>
      <h1>Lumexa Mission Control</h1>
    </header>
  );
}
```

```
// src/components/StatusPanel.jsx

// This component describes a small readout panel. Right now it always
// shows the same three values – Lesson 3 (useState) is where we make
// these values actually change over time.
export default function StatusPanel() {
  return (
    <section>
      <h2>Ship Status</h2>
      <ul>
        <li>Fuel: 82%</li>
        <li>Oxygen: 97%</li>
        <li>Hull Integrity: 100%</li>
      </ul>
    </section>
  );
}
```

```
// src/components/Footer.jsx

export default function Footer() {
  return (
    <footer>
      <p>Transmission relayed from Lumexa Station Alpha.</p>
    </footer>
  );
}
```

```
// src/App.jsx

// App is itself just a component – the "root" one. It composes the three
// smaller components together, the same way you'd nest <div> tags in HTML.
import Header from "../components/Header";
import StatusPanel from "../components/StatusPanel";
import Footer from "../components/Footer";

export default function App() {
  return (
    <div>
      <Header />
      <StatusPanel />
      <Footer />
    </div>
  );
}
```

```
// src/main.jsx (generated by Vite – shown here so you can see how App gets
// attached to the real page)
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "../App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Code Walkthrough

- **Each file is one component:** splitting `Header`, `StatusPanel`, and `Footer` into separate files keeps each piece focused on one job and easy to find later — this mirrors how real production React codebases are organized.
- **`export default function ComponentName()`**: this is the standard way to define and export a component so other files can `import` it.
- **Capitalized names:** `Header`, not `header` — React uses this capitalization convention to distinguish your custom components from built-in HTML tags in JSX.
- **`<Header />` self-closing tag**: when you use a component inside another component's returned UI, you write it like an HTML tag. Since it takes no children here, it's self-closed with `/>`.
- **`createRoot(...).render(<App />)`**: this is the one place where React "hands off" from plain JavaScript into the React world — it finds the real `<div id="root">` in `index.html` and tells React to render the `App`

component tree into it.

Student Mission

1. Add a fourth component, `AlertBanner`, that renders a `<div>` with the text `"All systems nominal."`, and include it inside `App` between `Header` and `StatusPanel`.
2. Add a second `<li>` to `StatusPanel` for `"Communications: Online"`.
3. Change the text inside `Footer` to include your own name, e.g. `"Transmission relayed from Lumexa Station Alpha, monitored by <your name>."`.

Challenge

Break `StatusPanel`'s list into its own separate component called `StatusList`, and have `StatusPanel` render `<StatusList />` instead of the `<ul>` directly. This is real refactoring practice: extracting a piece of one component into its own named component without changing what appears on the screen.

Experiment/Extension

Open your browser's DevTools Elements tab while `npm run dev` is running, and look at the actual HTML that ends up in the page. Notice there's no trace of "Header" or "StatusPanel" in the raw HTML tags — those names exist only in your source code and, more usefully, in the React DevTools extension (install it now if you haven't: search your browser's extension store for "React Developer Tools"). Open the new "🧩 Components" tab it adds to DevTools and see your component tree rendered by name.

Common Mistakes

- Naming a component starting with a lowercase letter (e.g. `function header()`) — React then treats `<header />` as an unknown HTML tag instead of your component, and it silently fails to render your code.
- Forgetting to `export default` a component, causing an "does not provide an export named default" error when another file tries to import it.
- Returning more than one top-level element from a component without wrapping them in a single parent (e.g. a `<div>` or a Fragment `<>...</>`) — JSX requires exactly one root element per return.
- Confusing a component *definition* (`function Header() { ... }`) with a component *usage* (`<Header />`) — defining it doesn't render it anywhere; you still have to place `<Header />` somewhere inside another component's returned JSX.

Debugging Tips

- Install the **React Developer Tools** browser extension — it adds "🧩 Components" and "🧩 Profiler" tabs to DevTools, letting you inspect your actual component tree, not just raw HTML.

- A blank white page with no visible error usually means check the browser Console first — React reports rendering errors there, often with a component stack trace showing exactly which component threw.
- The error "Adjacent JSX elements must be wrapped in an enclosing tag" means you returned two sibling elements without a shared parent — wrap them in a `<div>` or `<>...</>`.
- If your dev server won't start, check the terminal output first — Vite reports syntax errors (like a missing closing tag) with the exact file and line number.

Check Your Understanding

1. What is the difference between imperative and declarative UI code?
2. Why does React use a Virtual DOM instead of updating the real DOM directly for every change?
3. What naming rule must a component's function name follow, and why?
4. What does it mean to "compose" components together?
5. What command creates a new React + Vite project from scratch?

Mini Quiz + Answer Key

- Q1.** Which of these is a valid React component name? A) `header()` B) `Header()` C) `HEADER-component()` D) `2Header()`
- Q2.** What must every component's `return` statement produce? A) Any number of sibling elements B) Exactly one root element (or Fragment) C) A string only D) Nothing — return is optional
- Q3.** What is the Virtual DOM? A) A second, real DOM in a hidden browser tab B) An in-memory JavaScript description of the UI that React diffs against the previous version C) A CSS feature D) A database
- Q4.** Which tool is used to scaffold a new React project quickly? A) `npm create vite@latest` B) `git commit` C) `node --react` D) `npm audit`

Answer Key: 1-B, 2-B, 3-B, 4-A

Lesson Recap

You learned why React exists — to replace fragile, imperative DOM updates with a declarative model where components describe what the UI should look like for a given set of data. You created your first multi-component React app, saw how components compose together like nested building blocks, and set up a real Vite project.

Homework

Create a new Vite + React project called `crew-roster`. Build three components — `CrewHeader`, `CrewList` (a `<ul>` of at least four fictional crew member names), and `CrewFooter` — and compose them inside `App`. Run `npm run dev` and confirm it renders correctly in the browser.

Portfolio Project Connection

Projects 07, 08, and 09 are all structured as trees of small, focused components — a pattern you'll use constantly, starting with this lesson's `Header` / `StatusPanel` / `Footer` split.